

HPC

50 Sample Questions

Moderate introductory level

Instructions: Choose one correct answer (A–E) for each question. Assume valid inputs and successful execution unless stated otherwise. For numerical questions, use the assumptions given. This set emphasizes conceptual distinctions, short applications, calculations, and small code examples. Choose the best answer under the stated assumptions. There are 10 questions per topic, selected from the full bank. The answer key appears at the end.

HPC fundamentals

1. A simulation fits in one workstation's memory, but would take several months to finish. Which observation most strongly supports trying a parallel implementation?
 - A. Most of its computation can be divided into independent tasks
 - B. The input is stored in a single file
 - C. The sequential program already uses all available memory
 - D. The output contains many floating-point values
 - E. The source code contains many lines
2. Which description matches the basic distributed-memory view of a cluster?
 - A. All nodes share one physical RAM without communication
 - B. Each node is a core within one processor socket
 - C. Nodes have their own memory and exchange data over an interconnect
 - D. Each node is a thread with the same address space
 - E. Nodes exchange data only through a common CPU cache
3. Two MPI processes on different cluster nodes each declare `int x`. What is normally true of these variables in the basic message-passing model?
 - A. The same variable name makes them one shared object
 - B. They occupy separate process memory; one assignment does not update the other
 - C. A barrier automatically copies the updated value to the other process
 - D. The larger rank automatically sees the smaller rank's assignments
 - E. Running the same executable gives them the same variable storage
4. A processor applies one instruction to several data elements at once. Which form of parallelism is this?
 - A. Instruction-level parallelism across different instructions
 - B. Pipelining different stages of successive instructions
 - C. SIMD
 - D. Task parallelism across different functions
 - E. SPMD execution by independent processes

5. A simulation has 5×10^8 cells, performs 200 floating-point operations per cell per step, and runs for 10^4 steps. How many operations are required?
- A. 10^{15}
 - B. 5×10^{14}
 - C. 5×10^{15}
 - D. 10^{13}
 - E. 10^{14}
6. A processor has a high theoretical FLOP/s rate, but an array-processing program spends most of its time waiting for memory. Which conclusion is best supported?
- A. A low achieved FLOP/s rate proves an incorrect calculation
 - B. Peak arithmetic performance alone is insufficient to predict this program's speed
 - C. Peak FLOP/s determines runtime regardless of data access
 - D. Doubling arithmetic units must halve the runtime
 - E. Memory capacity alone determines the arithmetic throughput
7. A job waits 12 minutes in a queue and then executes for 8 minutes. What is its response time, measured from submission to completion?
- A. 16 minutes
 - B. 8 minutes
 - C. 12 minutes
 - D. 20 minutes
 - E. 24 minutes
8. Assume distinct array elements and no aliasing. Which loop is directly suitable for parallelizing its iterations?
- A. $a[i] = 2 * a[i-1]$ for increasing i
 - B. $x = 3 * x + 1$ using the preceding x
 - C. $\text{fib}[i] = \text{fib}[i-1] + \text{fib}[i-2]$ for increasing i
 - D. $a[i] = a[i-1] + 1$ for increasing i
 - E. $z[i] = x[i] + y[i]$ for each i
9. The best sequential version takes 120 seconds and a parallel version on 8 processors takes 20 seconds. What is the speedup?
- A. 4
 - B. 5
 - C. 8
 - D. 6
 - E. 10

10. Why can a parallel version run more slowly than a sequential version on a small input?

- A. The number of processors replaces execution time in the speedup formula
- B. Having fewer iterations per worker guarantees lower total runtime
- C. Idle workers increase useful arithmetic work by definition
- D. Dividing the data changes the sequential baseline to the sum of worker times
- E. Communication and synchronization overhead can exceed the computation saved

Memory and parallel design

11. A loop repeatedly accesses a small data set that remains in cache after its first pass. Why are later passes likely to spend less time waiting for data?

- A. Main memory becomes physically larger after the first pass
- B. More accesses can be served from cache instead of lower memory levels
- C. The cache permanently increases the clock frequency
- D. Each later pass necessarily executes fewer loop iterations
- E. The first pass eliminates the arithmetic in later passes

12. Why is a CPU cache useful even though it cannot hold all of main memory?

- A. Programs often reuse a limited set of recently or nearby accessed data
- B. Every program accesses all memory equally often
- C. Cache capacity automatically expands to match the working set
- D. Each data item is placed in a register for the entire program
- E. Using cache removes the need for data transfers from main memory

13. A program loads a value from main memory and immediately reads it again. Assume its cache line remains present. What is most likely for the second read?

- A. A hit only if the value has been modified since the first read
- B. A miss unless the address changes to a neighboring element
- C. A compulsory miss because it is a separate read instruction
- D. A disk access because the first load has finished
- E. A cache hit because the value was recently fetched

14. A program reads $a[0]$, $a[1]$, $a[2]$, and $a[3]$ consecutively. Which feature most directly helps this access pattern?

- A. Assigning each adjacent element to a different cache line
- B. Increasing the stride between consecutive accesses
- C. Keeping only the most recently used element rather than its neighbors
- D. Reusing the same individual element many times
- E. A cache line containing neighboring array elements

- 15.** Each matrix element is read ten times in independent calculations, and reordering is valid. Which organization best exploits temporal locality?
- A. Complete the ten uses of an element before moving to the next element
 - B. Perform five full scans, then another five full scans
 - C. Scan the full matrix once for each of the ten calculations
 - D. Complete each calculation across all elements before starting the next
 - E. Visit each element once before returning to any element
- 16.** A designer has identified small tasks and their communication, then combines neighboring tasks before assigning them to processors. Which design step is the combination?
- A. Mapping
 - B. Agglomeration
 - C. Communication identification
 - D. Load measurement
 - E. Partitioning
- 17.** What is a principal reason to agglomerate many small tasks?
- A. Increase scheduling opportunities by making each task still smaller
 - B. Remove data dependencies by changing processor assignments
 - C. Reduce communication and scheduling overhead relative to useful computation
 - D. Improve balance by forcing exactly one task per processor
 - E. Reduce useful arithmetic by assigning the same tasks to all processors
- 18.** A simulation divides its spatial grid into regions and applies the same update to each region. Which approach is this?
- A. Agglomeration of different functions
 - B. Dynamic mapping
 - C. Functional decomposition
 - D. Domain decomposition
 - E. Collective communication
- 19.** Independent Monte Carlo trials need no communication while generating samples, but their counts must be combined at the end. What pattern is appropriate?
- A. Tasks need repeated all-to-all exchanges of intermediate state
 - B. Tasks share a counter that must be updated at every step
 - C. Each task depends on the preceding task's intermediate value
 - D. Independent local computation followed by a global reduction
 - E. Tasks require a collective exchange before every arithmetic operation

- 20.** Sending many very small messages takes longer than sending the same total data in a few larger messages. Which cost most directly explains this?
- A. Each small message requires all computations to be repeated
 - B. Bandwidth is measured in messages rather than bytes per second
 - C. The startup latency paid for each message
 - D. The total payload becomes larger when divided into messages
 - E. The receiver's clock frequency is determined by message count

MPI

- 21.** Each process has one local maximum, and every process needs the global maximum. Which single collective is most appropriate?
- A. MPI_Allreduce with MPI_MAX
 - B. MPI_Bcast of rank 0's local maximum
 - C. MPI_Allgather of all local maxima
 - D. MPI_Reduce with MPI_MAX
 - E. MPI_Allreduce with MPI_SUM
- 22.** What does the SPMD model mean in a typical MPI program?
- A. Processes run the same program but can take different branches and operate on different data
 - B. All processes execute the same instruction at the same instant
 - C. Each data element must be processed by every rank
 - D. Each process must run a different executable
 - E. All ordinary variables have shared storage across ranks
- 23.** A process belongs to two different communicators. Which statement is correct?
- A. Its rank is its physical core number in both communicators
 - B. It must have different ranks in both communicators
 - C. It may have different ranks in the two communicators
 - D. Its rank must equal the total number of its communicators
 - E. It must have the same rank in both communicators
- 24.** A communicator contains six processes. What are their ranks within that communicator?
- A. The physical core numbers, regardless of communicator membership
 - B. 0 through 5
 - C. 1 through 5
 - D. 0 through 6
 - E. 1 through 6

- 25.** Two processes have different ranks in `MPI_COMM_WORLD`. Does that alone show that they run on different physical nodes?
- A. Yes; all processes on one node share a rank
 - B. No; different ranks must share one address space
 - C. Yes; each rank identifies a separate node
 - D. No; rank determines thread count instead of process identity
 - E. No; different ranks can run on the same node
- 26.** A rank contributes a local sum to `MPI_Reduce` with root 0. Which rank is guaranteed to receive the combined result in its receive buffer?
- A. All ranks except rank 0
 - B. Only the rank that enters the reduction last
 - C. Only the rank with the largest local sum
 - D. Every rank
 - E. Only rank 0
- 27.** In `MPI_Send(buffer, 5, MPI_DOUBLE, ...)`, what does the count 5 represent?
- A. Five bytes regardless of datatype
 - B. Five bytes for each process in the communicator
 - C. Five separate messages
 - D. Five elements of type `MPI_DOUBLE`
 - E. Five destination processes
- 28.** `MPI_Test` returns with its completion flag false. What does this tell the application?
- A. The operation must be restarted with a new send
 - B. The request has been cancelled
 - C. The request was not complete when tested; buffer reuse must still respect the pending operation
 - D. All processes must call `MPI_Test` simultaneously
 - E. The request has completed but the data is invalid
- 29.** Four ranks each contribute two integers to `MPI_Gather`. What is the minimum root receive-buffer capacity in integers for these contributions?
- A. 2
 - B. 8
 - C. 6
 - D. 16
 - E. 4

- 30.** Ranks 0 through 3 contribute 2, 4, 6, and 8 to MPI_Reduce with MPI_SUM and root 0. What result is placed in the root's receive buffer?
- A. 10
 - B. 12
 - C. 384
 - D. 20
 - E. 8

OpenMP and hybrid programming

- 31.** A shared array fits within one node, and independent iterations update different elements. Which approach most directly uses the node's cores without first dividing the array among separate processes?
- A. An OpenMP critical region around the whole loop executed by every thread
 - B. An OpenMP parallel for over the independent iterations
 - C. An MPI broadcast that automatically assigns array iterations to cores
 - D. An MPI barrier around the unchanged serial loop
 - E. A plain OpenMP parallel region that makes every thread execute all updates
- 32.** An OpenMP loop uses a temporary variable declared outside the parallel region. Every iteration assigns it before using it. Which change most directly avoids conflicting accesses without serializing the loop?
- A. Use static scheduling while keeping the temporary shared
 - B. Make the temporary private to each thread
 - C. Initialize the shared temporary to zero before the loop
 - D. Make the temporary shared explicitly
 - E. Place a barrier before the loop only
- 33.** What happens when a thread encounters a typical OpenMP parallel region?
- A. Each thread automatically becomes an MPI process
 - B. The team continues executing all later serial statements in parallel
 - C. A team executes the region, synchronizes at its end, and the encountering thread continues
 - D. Each statement is automatically assigned to exactly one team member
 - E. Only the primary thread executes the region while others wait

- 34.** What does an OpenMP for work-sharing construct do?
- A. Assign iterations according to their numerical results
 - B. Execute the entire loop on the first thread to arrive
 - C. Have every thread execute all iterations
 - D. Create a new team for each iteration
 - E. Divide loop iterations among the existing team of threads
- 35.** Several threads execute `sum += value` on the same shared integer without synchronization. What is the central problem?
- A. The runtime orders all additions by thread number
 - B. The shared clause gives each thread a private copy
 - C. The update is safe whenever each thread has a different value
 - D. Static scheduling makes the shared update indivisible
 - E. The updates can race because the read-modify-write operations are not protected
- 36.** Before a parallel region, `x` is 7. Which clause gives each thread its own `x` initially equal to 7?
- A. `firstprivate(x)`
 - B. `lastprivate(x)`
 - C. `reduction(+:x)`
 - D. `shared(x)`
 - E. `private(x)`
- 37.** A loop has iterations 0 through 11, four threads, and `schedule(static,2)`. Which iterations are assigned to thread 1?
- A. 1, 5, 9
 - B. 0, 1, 8, 9
 - C. 2, 6, 10
 - D. 2, 3, 10, 11
 - E. 3, 4, 5
- 38.** Why can a correct floating-point parallel reduction differ slightly from a sequential sum?
- A. Each thread includes every input value, increasing the total
 - B. The runtime rounds each thread's result to an integer
 - C. The addition order can change, and floating-point addition is not exactly associative
 - D. The reduction changes the declared precision of the variable
 - E. Different thread IDs directly change the input values

39. Assume valid OpenMP execution. What is sum after this code?

```
int sum = 0;
#pragma omp parallel for reduction(+:sum)
for (int i = 1; i <= 4; ++i)
    sum += i;
```

- A. 8
- B. 6
- C. 16
- D. 10
- E. 4

40. A cluster application uses MPI between nodes and OpenMP within each process. What is one potential advantage over keeping many separate copies in MPI processes on a node?

- A. Threads can share suitable data and reduce replication within each process
- B. All process-private arrays become globally shared automatically
- C. The serial fraction disappears when both models are used
- D. MPI messages become unnecessary between nodes
- E. Every hybrid configuration must outperform an MPI-only configuration

MapReduce and distributed storage

41. Which task is directly suited to a single basic MapReduce job?

- A. Compute total sales per product from many independent transaction records
- B. Modify one shared mutable record after every individual input record
- C. Update a state through a strict chain where each step needs the previous result
- D. Exchange grid-boundary values repeatedly within every map call
- E. Perform each iteration only after a global convergence check from the previous iteration

42. Two mappers emit (apple,2) and (apple,3). What grouped input should the sum reducer receive for apple?

- A. Key apple with the single value [5]
- B. Key apple with values [2,3]
- C. Key 2 with value apple and key 3 with value apple
- D. Key apple with values [1,1]
- E. Two independent reduce groups, one with [2] and one with [3]

- 43.** A mapper summarizes [2,4] and another summarizes [10]. To compute the overall mean correctly through local aggregation, what should each emit for the same key?
- A. Only its sum, followed by division by the number of mappers
 - B. Only its mean, followed by an unweighted average of mapper means
 - C. Only its count, followed by division by the number of mappers
 - D. A pair containing its sum and count
 - E. Its mean and minimum, ignoring the number of observations
- 44.** A mapper emits records with keys A, B, and A. In ordinary per-key reduction, which grouping property must hold?
- A. All A and B records must share one reducer regardless of partitioning
 - B. Mapper identity must determine grouping instead of the intermediate key
 - C. Every distinct key needs a dedicated physical worker
 - D. Every A record must be processed by a different reducer
 - E. Both A records reach the same reducer, which may also handle B
- 45.** Which responsibility normally belongs to the MapReduce execution framework?
- A. Choosing the meaning of application-specific keys
 - B. Choosing the business interpretation of output values
 - C. Scheduling tasks and recovering failed worker tasks
 - D. Choosing the application's aggregation formula
 - E. Deciding which input records are relevant to the analysis
- 46.** A system expands from 4 workers to 8 similar workers while retaining distributed execution. Which change has occurred?
- A. Replicating output files without adding compute workers
 - B. Increasing only the number of threads on an unchanged machine
 - C. Horizontal scaling by adding machines
 - D. Vertical scaling by upgrading one machine
 - E. Reducing data size while retaining the same hardware
- 47.** What is the primary purpose of a combiner in a word-count job?
- A. Combine all global counts before any mapper finishes
 - B. Choose which reducer owns each key
 - C. Guarantee that each word occurs on only one mapper
 - D. Sort the final output across all reducers
 - E. Aggregate local counts to reduce intermediate network traffic

- 48.** For a basic inverted index, what should the mapper emit for word w found in document d ?
- A. $(1, w)$
 - B. (w, d)
 - C. $(d, 1)$
 - D. (d, w)
 - E. $(w, 1)$
- 49.** An HDFS client knows a filename but needs to locate its blocks. Which component normally supplies block-location metadata?
- A. The word-count reducer
 - B. The MapReduce partitioner
 - C. The map-side combiner
 - D. The NameNode
 - E. The DataNode holding the first input record
- 50.** After obtaining block locations, where does an HDFS client normally get the actual file-block bytes?
- A. Directly from the relevant DataNodes
 - B. From the scheduler that allocated the job
 - C. From the mapper responsible for the filename
 - D. Through the NameNode, which relays every block
 - E. From the reducer assigned to the file's first key

Answer key

Q	Answer	Q	Answer	Q	Answer	Q	Answer	Q	Answer
1	A	11	B	21	A	31	B	41	A
2	C	12	A	22	A	32	B	42	B
3	B	13	E	23	C	33	C	43	D
4	C	14	E	24	B	34	E	44	E
5	A	15	A	25	E	35	E	45	C
6	B	16	B	26	E	36	A	46	C
7	D	17	C	27	D	37	D	47	E
8	E	18	D	28	C	38	C	48	B
9	D	19	D	29	B	39	D	49	D
10	E	20	C	30	D	40	A	50	A